ABL Customer Statement Intelligence Suite

Final Implementation, Reliability & Deployment Report

Document Type	Final Implementation & Deployment Report
Prepared By	Daniyal Saqib
Role	IT Intern (ABIP), Allied Bank Limited
Sponsor / Reviewer	Sir Affan Wahid
Project Duration	27 Aug 2026 - 10 Sep 2026 (15 calendar days)
Completion Date	10 Sep 2026
Application	App 1 of 2
Status	**FINAL - Production Functional**

Modules Delivered

- **Statement Intelligence** - upload and analyze synthetic CSV bank statements; answer exact and open-ended questions.
- **Recurring Payment Analysis** - detect repeated outgoing-description patterns as recurring payment candidates.
- **ABL Policy Assistant** - retrieval-augmented Q&A; over a curated corpus of public Allied Bank information, with source links.

Final project position: The application was completed ahead of the original 25 Sep window and was production-validated on 10 Sep 2026.

Live frontend: https://statement-intelligence-suite-abl.vercel.app

Backend: https://pure-temple-45004-09958cbb6652.herokuapp.com

1. Purpose, Scope & Data Safety

The suite demonstrates how deterministic financial-data processing, retrieval-augmented generation (RAG), semantic search and an LLM can be combined in a banking-oriented web application while keeping the demonstration isolated from live banking systems.

In scope

- Natural-language Q&A; over uploaded synthetic CSV statement data.
- Deterministic calculation of key banking facts such as spending, credits, balances, monthly totals and transaction counts.
- Recurring-payment candidate detection from repeated outgoing transaction descriptions.
- RAG-based Q&A; over a curated set of public Allied Bank policy/document chunks.
- Production deployment using Vercel, Heroku and PostgreSQL + pgvector.

Hard data constraints

- No connection to live Allied Bank customer accounts or core-banking systems.
- No confidential or internal Allied Bank documents are used by the Policy Assistant.
- Statement data used for development and demonstration is synthetic.
- Uploaded statement data is processed for the active workflow and is not stored in the persistent policy vector database.

The Policy Assistant is designed to fail closed: when retrieval does not produce a sufficiently relevant public-policy match, it returns an explicit 'could not find relevant information' response instead of fabricating an Allied Bank policy.

2. Final System Architecture

The final architecture separates exact financial computation from language generation. Python owns authoritative financial facts; Groq is used for qualitative interpretation and grounded policy answers.

```
USER BROWSER | v Vercel - React + Vite + Tailwind | v Heroku - FastAPI | +--> Statement
path | Parser -> deterministic facts -> Groq qualitative layer | +--> Recurring path |
normalized debit descriptions -> recurring candidates | +--> Policy path FastEmbed /
MiniLM -> PostgreSQL + pgvector -> relevance filter -> Groq -> answer + ABL sources
```

3. Final Technology Stack

Layer	Final Technology	Reason
Frontend	React.js + Tailwind CSS	Interactive component-based UI with fast utility-first styling.
Build Tool	Vite	Lightweight React development and production build tool.
Frontend Hosting	Vercel	Git-connected production frontend deployment.
Backend	FastAPI / Python 3.12	Typed REST APIs, validation and straightforward AI/ML integration.
Backend Hosting	Heroku	Production web dyno and managed deployment workflow.
LLM Provider	Groq	Low-latency language generation for statement interpretation and policy Q&A.;
LLM Model	openai/gpt-oss-20b	Final production model, configured with low reasoning effort for this workload.
Embeddings	FastEmbed 0.8.0	ONNX-based embedding runtime chosen to reduce memory footprint.
Embedding Model	sentence-transformers/all-MiniLM-L6-v2 (384D)	Free/local semantic embeddings compatible with pgvector vector(384).
Database	PostgreSQL	Persistent policy corpus metadata and relational storage.
Vector Search	pgvector	Similarity search directly inside PostgreSQL.

Why the stack changed during implementation

- **Gemini -> Groq:** Gemini availability/free-tier quota behavior caused reliability problems during production testing. The final application uses Groq with GPT-OSS 20B.
- **SentenceTransformers/PyTorch -> FastEmbed:** the previous embedding runtime contributed to Heroku memory pressure. FastEmbed/ONNX kept the same 384D MiniLM embedding model while substantially reducing runtime memory.
- **LLM arithmetic -> deterministic Python:** exact financial totals and comparisons are now computed by Python; the LLM is not treated as the source of truth for banking arithmetic.

4. Statement Intelligence

The statement workflow validates and parses uploaded CSV data into structured transactions. Exact/common questions are answered deterministically. Open-ended or compound questions use verified backend facts as authoritative context and allow the LLM to add only grounded interpretation.

- Current core metrics: transaction count, debit/credit totals, opening/closing balances, monthly spending/credits, merchant spending and largest debit/credit.
- Compound questions are prevented from being prematurely short-circuited by single-metric deterministic answers.
- When verified numeric figures are required, Python guarantees them in the final response. Numeric/currency output from the LLM can be discarded rather than risk presenting an incorrect financial value.

5. Recurring Payment Analysis

The final recurring-payment feature intentionally returns **candidates**, not guaranteed subscriptions. Debit transactions are grouped by normalized description; a description appearing more than once is returned as a recurring candidate.

Output	Meaning
description	Normalized repeated outgoing merchant/payee description.
occurrences	Number of matching debit transactions.
amounts	Observed debit amounts for the repeated description.
dates	Observed transaction dates for the repeated description.

Important limitation: repeated merchants such as grocery stores can be identified even when they are not formal subscriptions. This behavior is deliberate and is labeled as candidate detection in the interface.

6. ABL Policy Assistant

The Policy Assistant uses a curated demonstration corpus of seven public Allied Bank document chunks. Each question is embedded locally with all-MiniLM-L6-v2, compared against vector(384) embeddings in PostgreSQL + pgvector, and filtered before any LLM answer is generated.

- Embedding runtime: FastEmbed / ONNX.
- Embedding model: all-MiniLM-L6-v2, 384 dimensions.
- Retrieval: top 3 policy chunks.
- Relevance rule: semantic distance <= 0.75.
- Fail-closed behavior when no relevant policy chunk passes the threshold.
- Returned answers include visible public Allied Bank source links.

Current demonstration corpus

- Account and Electronic Banking Terms
- Changes to Terms and Conditions
- Financial Consumer Protection Framework
- Customer Complaints and Confidentiality
- Unclaimed Deposit Refund
- Unclaimed Deposit Required Documents
- Schedule of Charges

7. Reliability & Production Hardening

Risk / Failure Mode	Final Mitigation
Provider quota / availability	Migrated final LLM path to Groq; provider exceptions are converted to controlled 502 responses.
Incorrect LLM financial arithmetic	Authoritative values are computed in Python and used as verified facts; exact numeric output is guarded.
Heroku memory quota (R14)	Replaced PyTorch-heavy embedding runtime with FastEmbed/ONNX; current production release was tested without new R14 errors.
Weak policy match	Enforced relevance threshold and fail-closed response.
Duplicate policy reseeding	Final vector-store replacement workflow prepares embeddings then replaces the corpus transactionally.
Statement format variation	Parser hardened with aliases, BOM handling, delimiter/currency/date cases and explicit rejection of ambiguous schemas.

8. Validation Completed on 10 Sep 2026

- **38/38 automated backend tests passed** at the final Git checkpoint.
- Python compile check passed.
- Dependency consistency check passed.
- Git diff validation passed before commit.
- Production statement upload and deterministic Q&A; passed.
- Production guarded open-ended statement Q&A; passed.
- Production month filtering passed.
- Production recurring-payment analysis passed.
- Production Policy RAG + sources passed.
- Production policy database verified at exactly 7 rows.
- No R14 memory errors were detected for the current production release during final regression.

Final Git checkpoint

64c6189 - Stabilize Groq agents and optimize policy embeddings

9. Completion Statement

The ABL Customer Statement Intelligence Suite reached its final production-functional milestone on **10 September 2026**, ahead of the originally planned 25 September project window.

At completion, the Vercel frontend and Heroku backend were successfully integrated and demonstrated across all three modules: Statement Intelligence, Recurring Payments and the ABL Policy Assistant.

Delivered success criteria

- A synthetic demo statement can be uploaded, parsed and analyzed.
- Exact statement questions are answered deterministically.
- Open-ended statement analysis is grounded in backend-verified facts.
- Recurring outgoing-payment candidates are detected and summarized.
- Relevant public ABL policy questions return grounded answers with sources.
- Out-of-domain policy questions fail closed instead of fabricating answers.
- The production application is reachable through the live Vercel URL.

Known MVP boundaries

- Demonstration data only; no live customer/account integration.
- Statement input is CSV-oriented; production-grade PDF/XLSX ingestion is outside this delivery.
- Recurring-payment results are candidates, not guaranteed subscription classifications.
- Policy knowledge is limited to the curated public corpus and is not a complete representation of all Allied Bank policies.

Project Status: COMPLETED - 10 SEP 2026